Exercício 004

1) Importância da Preparação

A qualidade do dado precede a eficácia do algoritmo (GIGO - Garbage In, Garbage Out).

* Viés: O modelo aprende ruídos e correlações falsas
* Instabilidade: Escalas diferentes distorcem a importância das variáveis.
* Ineficiência: Algoritmos demoram mais para convergir ou falham em encontrar o ótimo global.

2) Valores ausentes (Dado)

a) Estratégias:

* Imputação: Substituir por média (distribuição normal) ou mediana (presença de outliers).
* Predição: Usar outros atributos para estimar o valor faltante via regressão ou KNN.
* Sinalização: Criar uma coluna binária indicando que o dado era faltante.

b) Inadequação da Remoção:

* Quando a ausência não é aleatória (MNAR), gerando um modelo enviesado.
* Quando a perda de 25% reduz drasticamente o poder estático da amostra.

3) Normalização VS. Padronização

* Normalização (Min-Max): Redimensiona para [0,1]. Ideal para KNN e redes neurais, pois equaliza o peso de variáveis em cálculos de distância.

* Padronização (Z-Score): Ajusta para média 0 e desvio 1. Recomendada para Regressão linear e PCA, sendo mais resiliente a valores discrepantes.

4) Dados Categóricos (Clima)

a) Como transformar: Utilizando One-Hot Encoding. Cria-se uma coluna binária para cada categoria (ex: is_sol, is_chuva, is_nublado), onde 1 indica presença e 0 ausência.

b) Problema da codificação simples (0,1,2): O modelo interpretará uma hierarquia ou ordem inexistente (ex: entender que "nublado" é mais ou melhor que "sol" porque 2>0), o que induz o algoritmo de erro em variáveis nominais.

5) Overfitting e Separação de dados

Porque dividir:

* Treino: Para o modelo aprender os padrões e pesos
* Validação: Para ajustar hiperparâmetros e evitar overfitting durante o desenvolvimento.
* Teste: Para avaliar a capacidade de generalização em dados reais e inéditos.

Uso do teste no treinamento:

Ocorre o Data leakage (vazamento de dados). O modelo "decora" as respostas finais, resultando em métricas de desempenho falsamente altas e falha total ao lidar com dados novos no futuro.

6) Balanceamento de classes (95% A / 5% B)

a) Problema: O modelo sofrerá de viés majoritário. Ele pode atingir 95% de acurácia simplesmente prevendo "Classe A" para todos os casos, ignorando completamente a classe minoritária (B), que geralmente é a mais importante (ex: detecção de fraudes ou doenças).

b) Técnicas de tratamento:

1. Oversampling (Smote): Criar dados sintéticos para a classe minoritária (B).

2. Undersampling: Reduzir aleatoriamente a quantidade de amostras da classe majoritária (A).

7) Detecção de outliers

a) O que são: Valores atípicos que se distanciam drasticamente dos demais observações do dataset, podendo indicar erros de coleta ou eventos raros.

b) Quando a remoção prejudica: Quando o outlier representa um evento legítimo e importante para o negócio. Em detecção de fraudes ou falhas em máquinas, o outlier é exatamente o que o modelo precisa aprender identificar.

c) Método estatístico: Z-Score (considera outliers valores com |Z| > 3) ou IQR (amplitude interquartílica), que usa Boxplot para identificar valores fora dos limites inferior e superior.

8) Feature Engineering (data_nascimento)

Criar a variável idade é superior porque:

1. Redução de complexidade: A idade é um valor numérico escalar direto, enquanto a data exige que o modelo aprenda a aritmética de tempo.

2. Poder preditivo: A idade captura o fenômeno (ex: perfil de consumo) de forma estática, enquanto a data de nascimento exigiria que o modelo fosse atualizado constantemente para entender que o tempo passou.

9) Pipeline de pré-processamento

Um fluxo lógico e eficiente seria:

1. Tratamento de valores ausentes: Imputação (média/mediana) para não perder dados nas etapas seguintes.

2. Codificação de dados categóricos: One-Hot encoding ou label encoding para converter textos em números.

3. Escalonamento (Normalização / padronização): Colocar variáveis em escalas comparáveis para algoritmos sensíveis a distância

Ygrafia:

4. Balanceamento de classes: Aplicar Smote ou undersampling apenas no conjunto de treino para evitar viés majoritário.

10) Estudo de caso aplicado

1. Limpeza e Imputação: Tratar valores nulos em renda_mensal e idade usando a mediana (para evitar distorções por salários muito altos/baixos)

2. Codificação categórica: Aplicar one-Hot encoding em estado_civil e converter historico_de_inadimplencia (sim/não) em binário (1/0).

3. Tratamento de Outliers: Verificar renda_mensal via métrica de IQR, pois rendas extremas podem desregular o modelo de crédito.

4. Engenharia de atributos: Criar uma variável de "Renda per dependente" $\left(\frac{\text{renda_mensal}}{\text{numero_de_dependentes}+1}\right)$ que costuma ser um preditor de risco mais forte que a renda isolada.

5. Escalonamento: Aplicar padronização (Z-score) em renda_mensal e idade, garantindo que a diferença de magnitude entre os valores (ex: 5.000 de renda vs 30 de idade) não engane o algoritmo.

6. Balanceamento: Como a aprovação costuma ser mais comum que a negação, aplicar Smote no conjunto de treino para que o modelo aprenda a identificar perfis de risco (classe minoritária).